Comparing CNN, Hybrid QCNN, Full QNN For High-Dimensional Data

1조 - 김동욱, 남소영, 서정욱, 이건

목차
Contents

01
PROJECT TASK

02
MODEL ARCHITECTURES & ACCURACY

03
COMPARING RESOURCES

04
CONCLUSION

Motivation：Why Quantum Circuits for High-Dimensional Data?

- High-dimensional inputs (like images) require expressive models that can capture local and global dependencies.
- Classical models need deep layers and many parameters to approximate such representations.
- Quantum circuits can embed and entangle local patch information into compact quantum states — potentially offering efficient and expressive feature extraction.

Project Task

- Image Classification
  - AI-gen Image VS Real Image Classification
  - AI-gen Image is made with Diffusion Model

- Using Quantum Neural Network(QNN)
  - Try Classic CNN, Hybrid QNN, and Full QNN
  - Compare the model size and utilization of resources

“AI 이미지를 활용한 컨텐츠가 늘어나는 지금,
QNN은 이를 잘 구분할 수 있을까?”

Project Task

- **Dataset**
  - **‘AI recognition dataset’, superpotato9, Kaggle**
- **AI-gen images**
  - **Consists largely of artistic images rather than realistic ones**
  - **Generated with AIs including DALL-E and Midjourney**
- **DALL-E 2 and Midjourney are known to use diffusion models**
  - **Turns noise into an image**

superpotato9, “AI recognition dataset”, kaggle, https://www.kaggle.com/datasets/superpotato9/dalle-recognition-dataset
OpenAI, “DALL·E 2 Preview - Risks and Limitations”, github, 2022, https://github.com/openai/dalle-2-preview/blob/main/system-card.md
“Seeds”, midjourney, https://docs.midjourney.com/hc/en-us/articles/32604356340877-Seeds
“What are diffusion models?”, IBM, 2024, https://www.ibm.com/think/topics/diffusion-models

Dataset

AI

Real

Dataset

- 기존 문제점
  - 편향적인 label의 개수
    - real-image : AI-gen-image 약 8:2
  - 과도한 image resolution의 downscaling
    - 1024x1024 → 64x64
    - 이미지 구별을 위한 특징이 이 과정에서 소실되었을 수 있음

- 개선 후
  - 하나의 이미지를 256개의 패치로 분할
  - label의 비율을 맞춤

Dataset

- 상세 개요
  - Num of Train Dataset : 27360
  - Ratio of Dataset
    - Train : Valid : Test = 8 : 1 : 1
  - Ratio of label
    - Real : AI-gen = 1 : 1
  - Image size :
    - 3 x 64 x 64

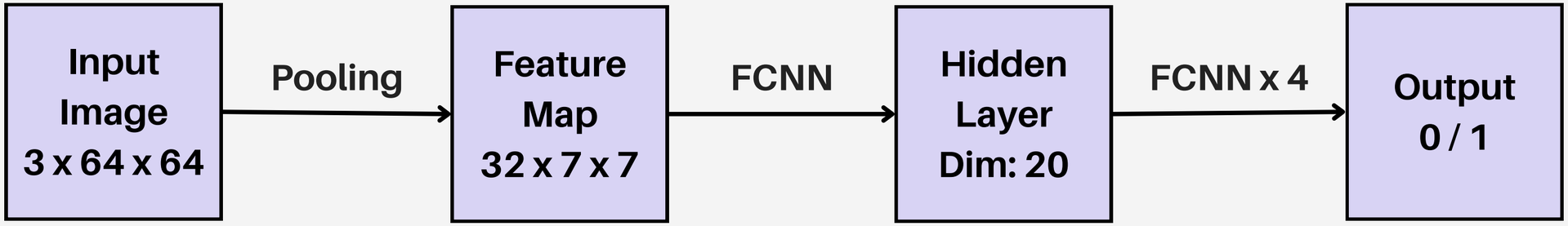
Previous Work

- 4 종류의 서로 다른 quantum circuit을 이용한 Hybrid QCNN 모델들의 성능 비교
  - 개선 전 Dataset 이용
- Model #1
  - Qiskit Aer Simulator 사용 (IBM Yonsei 시도)
  - 2 convolution layer - 2 FC layer - 2-qubit QC - FC layer 구조 (entanglement X)
  - Quantum circuit으로 1-length vector 전달
  - Accuracy : 67.0%
- Model #2
  - PennyLane 사용
  - pooling 이후 FC layer - 12 qubit QC - FC layer 구조 (entanglement O)
  - Accuracy : 49.65%
  - 구현 상에서의 오류로 인한 결과로 추정

Previous Work

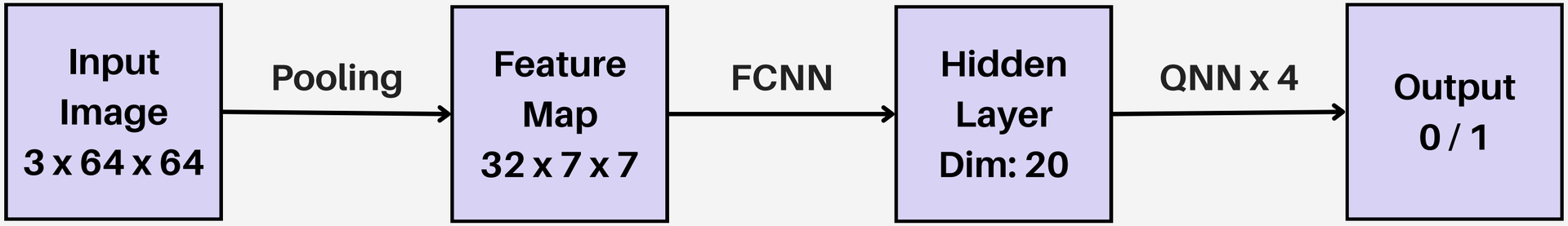
- Model #3
  - Qiskit Aer Simulator 사용 (IBM Brisbane 시도)
  - pooling 이후 FC layer - 1 qubit QC 구조 (entanglement X)
  - Accuracy : 78.0%
- Model #4*
  - PennyLane 사용
  - pooling 이후 FC layer - parallel 5-qubit QC - FC layer 구조 (entangled for 5 qubit)
  - Accuracy : 83.15%
  - 독립된 4개의 그룹으로 나눈 뒤, 각각에 대해 5 qubit QNN 구조로 학습
  - Baseline Hybrid QCNN으로 선정

Classical CNN



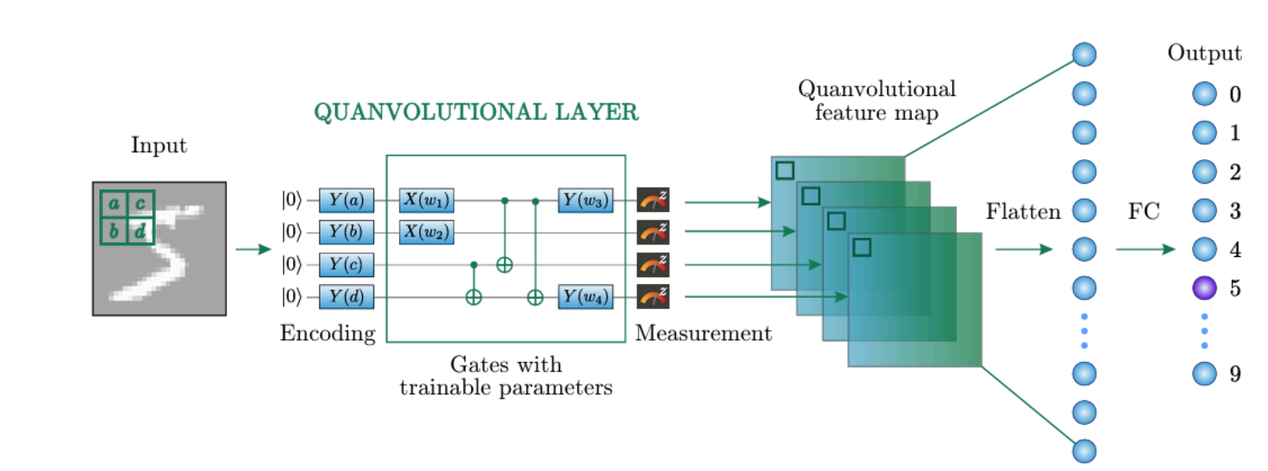
- 기본적인 형태의 image classification CNN
  - quantum 사용 x
- Hybrid QCNN과의 비교를 위해 4개의 FCNN으로 구성
- Pooling을 통한 특징 추출 → Fully Connected Neural Network

Hybrid QCNN1



- PennyLane 사용
- classical pooling 이후 QNN이 결합된 Hybrid 구조
  - 5-qubit 구조 : 4개의 독립된 QNN을 학습 / 총 20큐빗의 Weight 학습
- AngleEmbedding
  - 입력값을 Y축 회전으로 큐빗에 적용하는 회로
- BasicEntanglerLayers
  - 회전(Rx, Ry, Rz)과 얽힘(CNOT)으로 구성된 회로

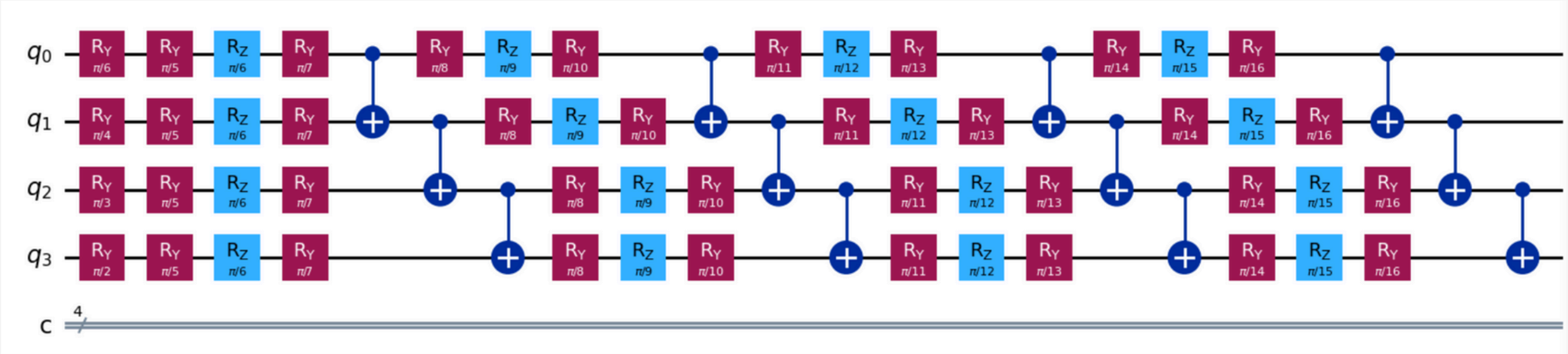
Hybrid QCNN2



Source: Senokosov et al., Quantum machine learning for image classification,
Mach. Learn.: Sci. Technol., 5, 015040 (2024). https://doi.org/10.1088/2632-2153/ad2aef

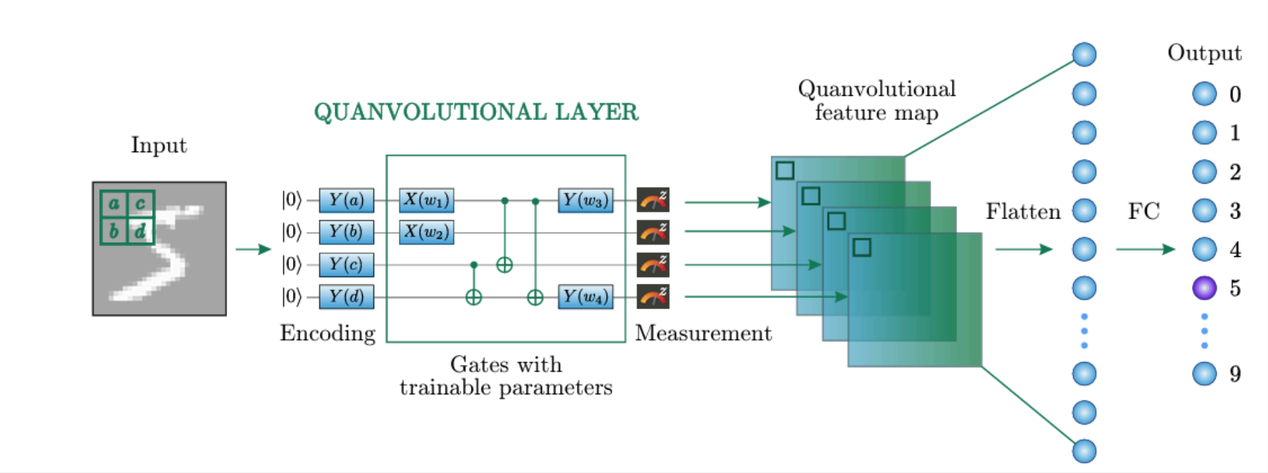
- PennyLane 사용 (다음 슬라이드의 회로 시각화는 Qiskit)
- Feature extractor : 2×2 patch → 4-qubit QC
  - Stride = 2, Padding = 0 → non-overlapping patches
  - Patch의 픽셀 값을 angle encoding하여 각 qubit에 적용

Hybrid QCNN2



- Quantum Circuit：angle embedding → variational part → Z-basis measurement
  - Angle embedding：Rotate the qubits from their initial state, i.e., |0>, with the Rotation-Y gate
  - variational part：single-qubit rotation gates on each qubit(Ry,Rz,Ry)
    → CNOT(0,1) CNOT(1,2) CNOT(2,3)
  - Z-basis measurement：measure +1 / -1
- # of parameters：4*4 =16

Hybrid QCNN2



Source: Senokosov et al., Quantum machine learning for image classification,
Mach. Learn.: Sci. Technol., 5, 015040 (2024). https://doi.org/10.1088/2632-2153/ad2aef

- Quantum Circuit : 각 qubit의 <Z>= P(|0〉)-P(|1〉)를 계산하여 4-length vector 를 반환
- 위 과정이 각 R, G, B channel에 대해서 개별적으로 진행됨
- 2 FC layers : C*4 - length vector → probs for binary class

Hybrid QCNN2

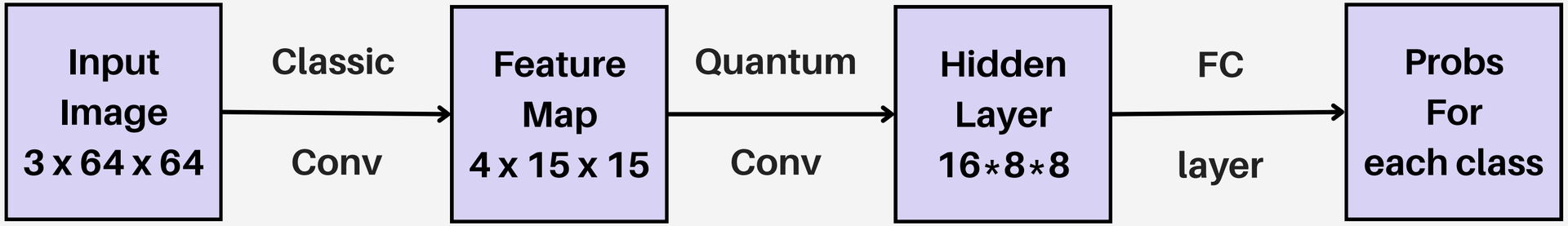
- Advantages
  - Feature extractor - Quantum Circuit → model Depth 대폭 감소
  - # of learnable parameters 대폭 감소
- Limitations
  - pennylane에서 제공하는 basic entanglement 구조를 변형 없이 사용함
  - Quantum Circuit의 반복 수: $C \times (\lfloor (H-2)/2 \rfloor + 1) \times (\lfloor (W-2)/2 \rfloor + 1)$
  - e.g. 64*64 RGB image : 3*32*32 = 3072 번 양자 회로 호출

    → 이미지 크기가 커질수록 양자 회로 호출 횟수 폭증

Hybrid QCNN2

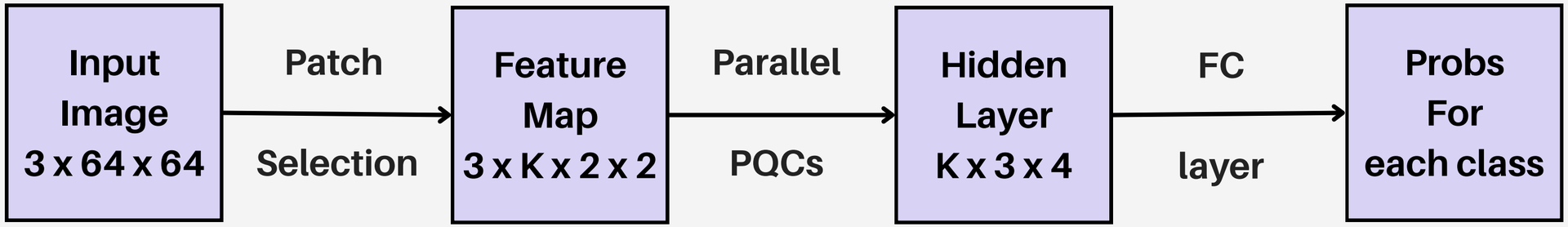
```
epoch=0, lr=0.001000, 2025-06-16 18:01:28.267881
Training [5%]   Loss: 0.6797
Train Loss: 0.6797, Train Acc: 57.00%
Val Loss: 0.6846, Val Acc: 59.00%
✅ Improved val_acc: 59.00%, best model saved.
epoch=1, lr=0.001000, 2025-06-16 18:44:04.408502
```

- **Model Variation**
  - **Conv → Quantum Bottleneck → Classifier**
    - **model structure:**
      - **Classical CNN for low-level features (4 * 4 kernel, stride=4, padding=0)**
      - **Quantum layer acts as nonlinear embedding (4*8*8 QC applications per image)**



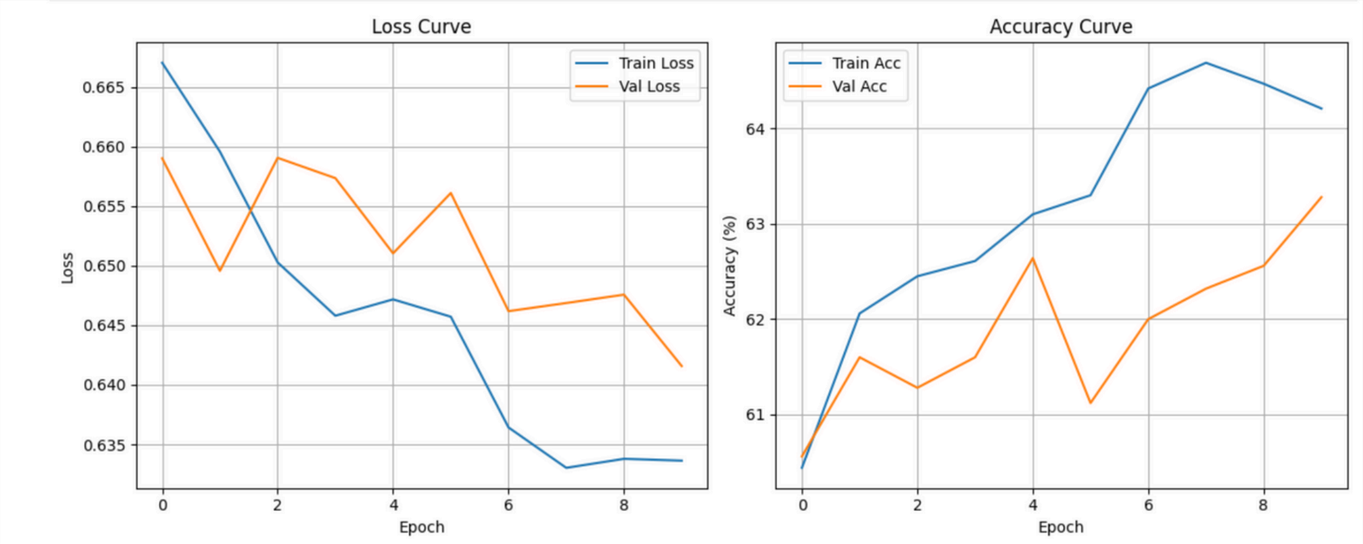
Hybrid QCNN2

- **Model Variation**
  - **Selecting Top K patches → Parallel PQC → Classifier**
    - model structure(depth = 3):
      - Selecting top K patches that have the greatest within-patch variation per channel
      - Apply parallel PQCs for each channel (K*3 QC applications per image)
      - 2 FC layers



Hybrid QCNN2

- **Model Variation**
  - **Selecting Top K patches → Parallel PQC → Classifier**



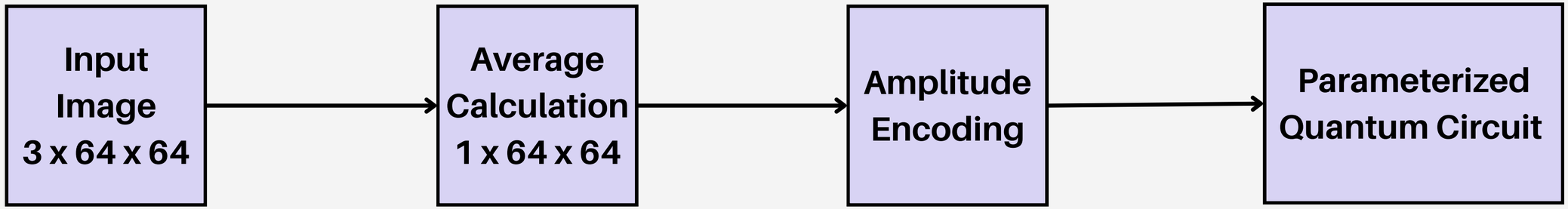
Test Accuracy 66.24%

# of training samples：10,000
3-depth CNN with the same data： Test Accuracy 63.60%

Hybrid QCNN2

- **Future Suggestions**
  - **PCA는 principal components의 수에 따라서 overfitting/underfitting 으로 이어지는 것을 확인함 즉, 이미지 데이터에서 잘 작동하지 않았음**
  - **따라서 UMAP을 이용한 차원 축소 방법을 활용해 볼 예정임**
  - **고차원 이미지 데이터에 가장 적합한 구조를 찾는 것을 목표로 다양한 entanglement structure을 적용해보려고 함**

Full QNN 1

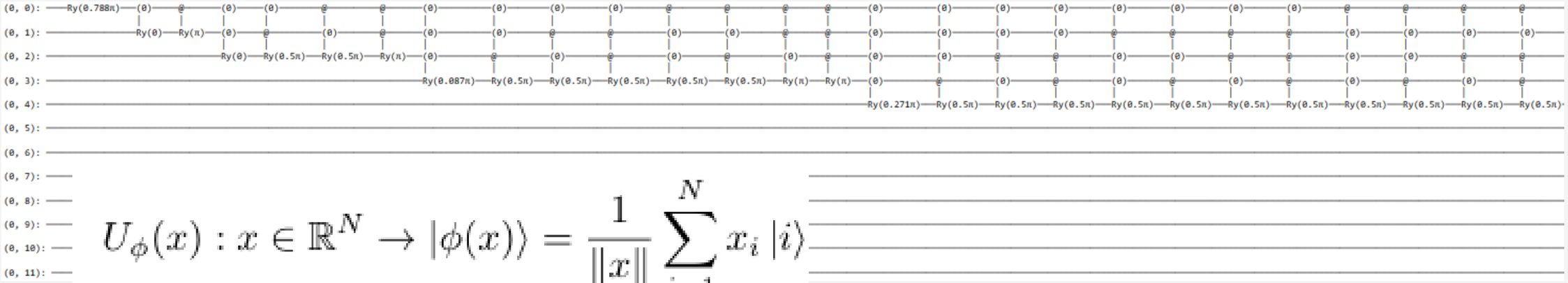


- **양자 시뮬레이터: Cirq, TensorFlow Quantum**
- **Code based on "milind-u/image-qcnn", <github>, 10-22-2022**
  - Upadhyay, M. (2022). *image-qcnn*, [GitHub repository]. https://github.com/milind-u/image-qcnn
- **Quantum Circuit layers based on**
  - **Lü, Y., Gao, Q., Lü, J., Ogorzałek, M., & Zheng, J. (2021). A quantum convolutional neural network for image classification.** Proceedings of the 40th Chinese Control Conference, 6329-6334. Shanghai, China: IEEE.
  - **Nielsen, M. A., & Chung, I. L. (2000). Quantum Computation and Quantum Information.** 61-73. New York: Cambridge University Press.

Full QNN 1

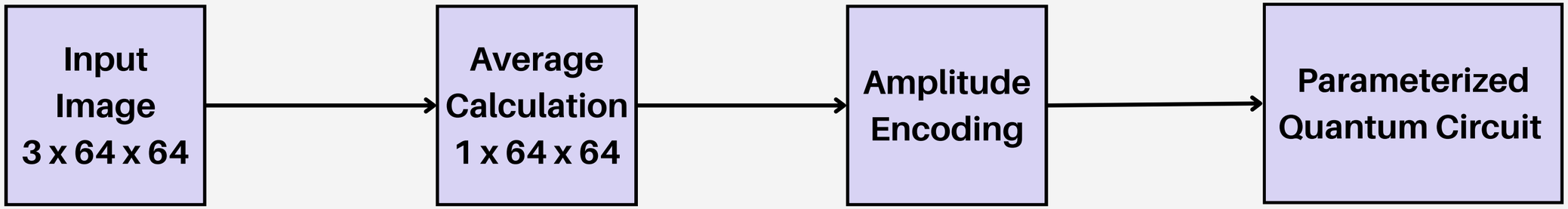
Input Image 3 x 64 x 64 → Average Calculation 1 x 64 x 64 → Amplitude Encoding → Parameterized Quantum Circuit

- **Amplitude Encoding**



$$U_\phi(x) : x \in \mathbb{R}^N \rightarrow |\phi(x)\rangle = \frac{1}{\|x\|} \sum_{i=1}^{N} x_i \, |i\rangle$$

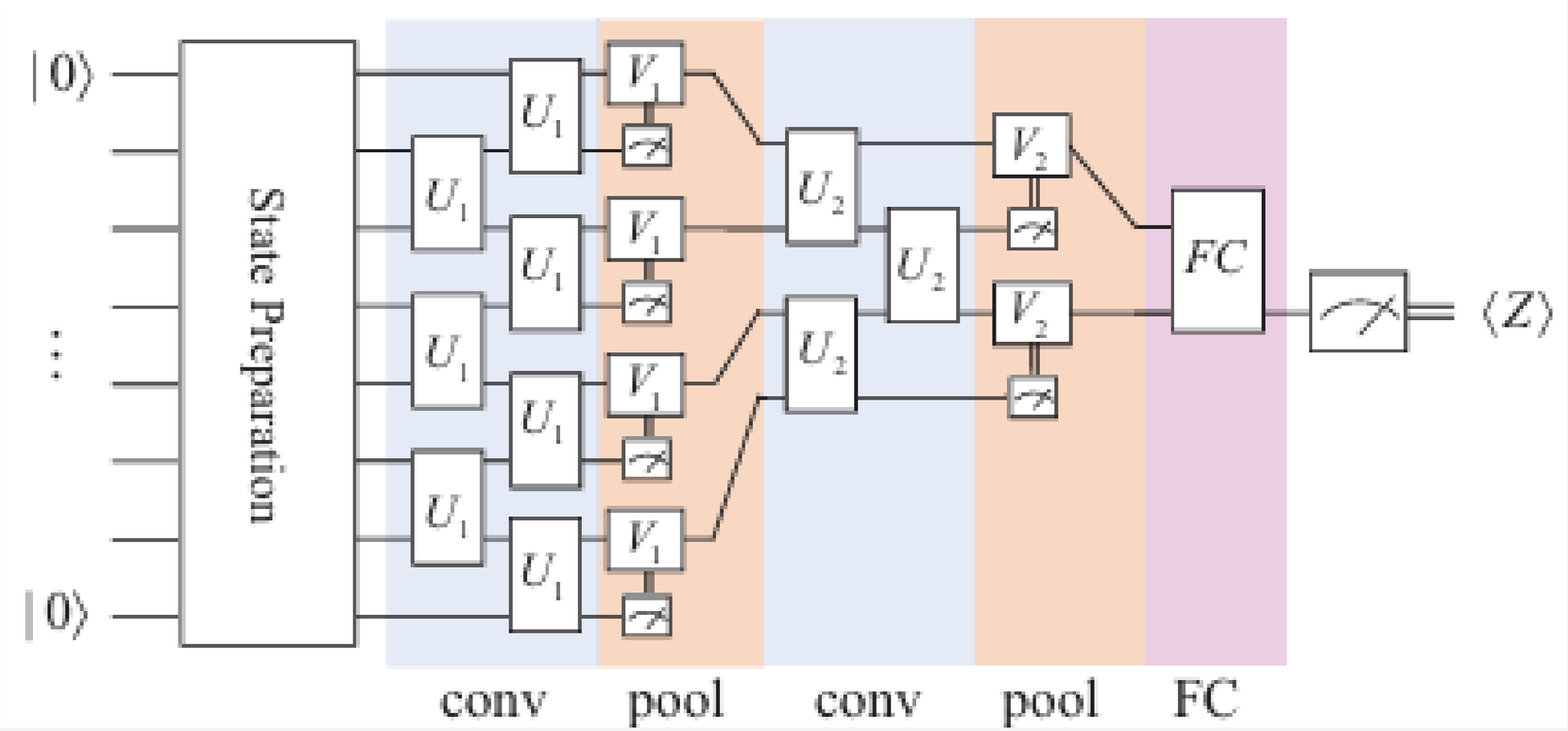
Full QNN 1



- **Amplitude Encoding**
  - Naive implementation은 시간, 공간적으로 매우 비효율적
  - Qubit 수에 대해 지수적인 양의 Controlled-Gate가 필요
- **Parameterized Quantum Circuit**
  - 12 qubits, 4 layers ($64^2 = 2^{12}$)
  - 마지막 Qubit만 측정

Full QNN 1

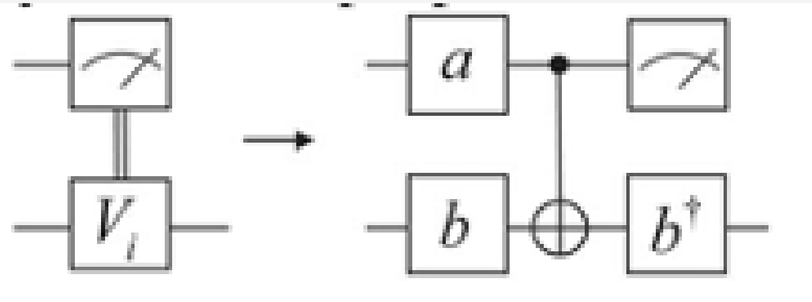
- **Parameterized Quantum Circuit**



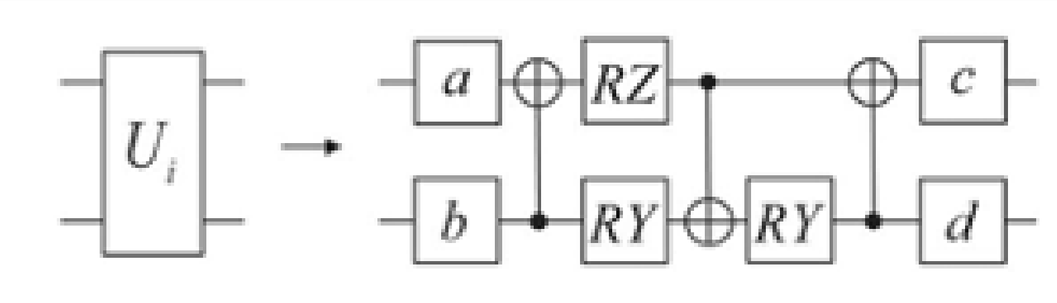
Lü, Y., Gao, Q., Lü, J., Ogorzałek, M., & Zheng, J. (2021). A quantum convolutional neural network for image classification. Proceedings of the 40th Chinese Control Conference, 6329-6334. Shanghai, China: IEEE.

Full QNN 1

- **Parameterized Quantum Circuit**
  - **Principle of deferred measurement**



  - **We can decompose any 1-qubit gate into Rz, Ry, Rz gates**



Diagrams from: Lü, Y., Gao, Q., Lü, J., Ogorzałek, M., & Zheng, J. (2021). A quantum convolutional neural network for image classification. Proceedings of the 40th Chinese Control Conference, 6329-6334. Shanghai, China: IEEE.

Full QNN 1

- Advantages
  - 다양한 형태의 Gate가 구현될 수 있음
  - Classic 단계를 거치지 않으면서 이미지 처리 가능
- Limitations
  - 인코딩에 대한 naive implementation의 사용은 시간, 공간적으로 비현실적
  - 충분히 유의미한 학습 효과가 관찰되지 않음
  - RGB를 흑백으로 변환하여 분석에 활용

Full QNN 2

- **PennyLane 사용**

Input Image 3 x 64 x 64 → **PCA(주성분 분석) 128 dim** → **Vector Transition** →

Amplitude Embedding

$$\vec{x} \in \mathbb{R}^N \rightarrow |\psi\rangle = \sum_{i=0}^{N-1} x_i |i\rangle$$

Quantum Data Encoding

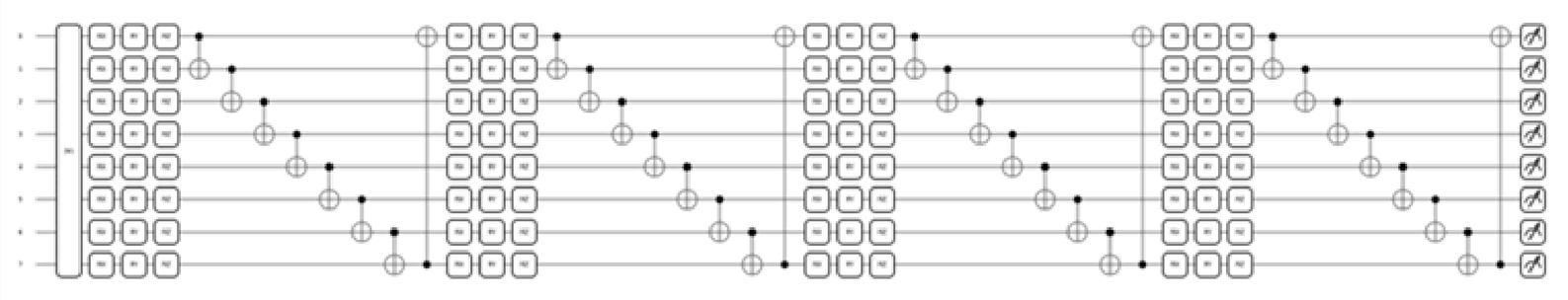
- Angle Embedding은 각 큐비트마다 하나의 입력값이 할당
→ 입력 벡터의 차원이 큐비트의 수에 의해 제한

N-dim vector → $\log_2 N$ qubits

하지만, Amplitude 임베딩을 하면

- 양자 상태 전체에 분포시킴
- 양자 간섭, 얽힘이 효과적으로 반영
- 이미지같은 연속 분포 데이터에서는
- 전체 벡터의 상대적 비율이 중요 → Amplitude 임베딩 더욱 효과적

Parameterized Quantum Circuit (PQC)



OUTPUT

- $< Z > = P(|0\rangle) - P(|1\rangle)$ 값들을 여러 큐비트에서 뽑아낸 후,
- 그걸 그대로 예측값(logit)으로 사용함.

Full QNN 2

- **Advantages**
  - PCA - Quantum Circuit → 모델 Depth 감소
  - 파라미터 수: n_layers × n_qubits × 3 (RX/RY/RZ)
- **Limitations**
  - No Spatial Feature Compression
    - CNN처럼 지역 receptive field가 없기 때문에, raw pixel 수준에서 바로 전역 특징 학습이 어려움
    - 이미지의 패턴(local structure)을 추출하는 데 비효율적
  - Input Dimension 제한
    - AmplitudeEmbedding은 입력 벡터 길이가 2^n이어야 하므로, 강제로 zero-padding or PCA 압축 필요
    - 데이터가 커질수록 억지로 자르거나 압축해야 해서 정보 손실 발생 가능

Result

- Classical CNN
  - Accuracy : 82.43%
- Hybrid QCNN1
  - Accuracy : 88.56%
- Full QNN 1
  - Accuracy : 50.92% (With Simplified Encoding)
- Full QNN 2
  - Accuracy: 49.3% (1 Epoch) → 55.6% (5 Epoch)

Classical CNN & Hybrid QCNN1

```
----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
            Conv2d-1           [-1, 16, 64, 64]           1,216
       BatchNorm2d-2           [-1, 16, 64, 64]              32
         MaxPool2d-3           [-1, 16, 32, 32]               0
            Conv2d-4           [-1, 32, 32, 32]          12,832
       BatchNorm2d-5           [-1, 32, 32, 32]              64
         MaxPool2d-6           [-1, 32, 16, 16]               0
 AdaptiveAvgPool2d-7             [-1, 32, 7, 7]               0
            Linear-8                  [-1, 120]         188,280
            Linear-9                   [-1, 20]           2,420
       TorchLayer-10                    [-1, 5]               0
       TorchLayer-11                    [-1, 5]               0
       TorchLayer-12                    [-1, 5]               0
       TorchLayer-13                    [-1, 5]               0
           Linear-14                    [-1, 2]              42
================================================================
Total params: 204,886
Trainable params: 204,886
Non-trainable params: 0
```

- **TorchLayer**
  - **CNN의 경우 : FCNN**
  - **QCNN의 경우 : 5-qubit QNN with entangled**

Full QNN 1

- 12 Qubits
- 4 layers
- Gates
  - Exponentially many on the number of qubits for encoding
  - 18 gates per convolutional unit
  - 10 gates per pooling unit
  - convolutional and pooling units O(nlogn)
- batch size: 50
- epochs: 5

Full QNN 2

Category	Value	Explanation
Number of Qubits (`n_qubits`)	7	Number of qubits in the quantum circuit
Number of PQC Layers (`n_layers`)	4	Number of repeated variational layers
Gates per Layer	28 gates	3 single-qubit gates (RX, RY, RZ) per qubit + 6 CNOTs + 1 wraparound CNOT
Total Gates per QNode Call	112 gates	28 gates × 4 layers
Batch Size (`batch_size`)	16	Number of samples processed in one forward pass
Training Samples	2000	Total number of training examples
Epochs	5	Total number of epochs
QNode Calls per Epoch	2000	Every training sample triggers a QNode call
Total QNode Calls	**10,000**	2000 samples × 5 epochs
Total Quantum Gate Executions	**1,120,000 gates**	10,000 calls × 112 gates per call

Conclusion

- QML과 classical machine learning을 비교함으로써 QML을 통해 상대적으로 적은 수의 parameter로 기존 classical model과 유사한 성능을 낼 수 있음을 확인함
- Hybrid QCNN1:
  - 실제 20qubit을 동시에 실행시키지 못하기에 5-qubit으로 구성된 4개의 그룹으로 분할
  - 학습이 병렬적으로 진행되어 5개의 qubit만 사용하여 학습 진행
  - 학습 시간/성능적인 측면에서도 경쟁력 있는 결과를 보임
- Hybrid QCNN2:
  - High-dimensional data에 대하여 Quantum layer를 feature extractor로 사용하는 것은 아직 어려움
  - 그러나 quantum layer와 classical layer를 적절히 결합할 경우, 동일한 깊이를 가지는 classical model에 비해 모델의 expressive power가 상당히 향상됨

Conclusion

- **Full QNN 2:**
  - **Quantum network를 돌리기 전에 feature 추출이나 데이터 압축이 매우 어려움 (이미지 패턴 분석에 불리)**
  - **Amplitude Embedding 방식은 2^n qubit의 차원만 입력받을 수 있기에 학습 과정에서 정보 손실이 컸음 (zero-padding이나 이미지 패치, PCA 등이 강제됨)**
  - **양자 시뮬레이터 자체가 성능이 좋은 cpu 혹은 실제 device가 아니면 속도가 느리다는 점도 학습 부진에 영향을 미침**

Reference

[1]Pontorno, O.. (2023). *Hybrid-Quantum-Neural-Network-for-binary-classification* [GitHub repository]. https://github.com/opontorno/Hybrid-Quantum-Neural-Network-for-binary-classification

[2]Senokosov, A., Sedykh, A., Sagingalieva, A., Kyriacou, B., & Melnikov, A. (2024). Quantum machine learning for image classification. Machine Learning: Science and Technology, 5(1), 015040. https://doi.org/10.1088/2632-2153/ad2aef

[3]Havlíček, V., Córcoles, A. D., Temme, K., Harrow, A. W., Kandala, A., Chow, J. M., & Gambetta, J. M. (2019). Supervised learning with quantum-enhanced feature spaces. Nature, 567(7747), 209–212. https://doi.org/10.1038/s41586-019-0980-2

[4]Rawat, A. S. (2024). *Quantum-Convolutional-Neural-Network* [GitHub repository]. https://github.com/AnkRaw/Quantum-Convolutional-Neural-Network

[5]Sehrawat, A. (2023). *Image_classification_with_CNN_and_QNN* [GitHub repository]. https://github.com/ArunSehrawat/Image_classification_with_CNN_and_QNN

[6]Lü, Y., Gao, Q., Lü, J., Ogorzałek, M., & Zheng, J. (2021). A quantum convolutional neural network for image classification. Proceedings of the 40th Chinese Control Conference, 6329-6334. Shanghai, China: IEEE.

Reference

[7]Henderson, M., Shakya, S., Pradhan, S., & Cook, T. (2019). Quanvolutional neural networks: Powering image recognition with quantum circuits. arXiv preprint arXiv:1904.04767. https://arxiv.org/abs/1904.04767

[8]Nielsen, M. A., & Chung, I. L. (2000). Quantum Computation and Quantum Information. 61-73. New York: Cambridge University Press.

[9]Upadhyay, M. (2022). *image-qcnn*, [GitHub repository]. https://github.com/milind-u/image-qcnn

[10]superpotato9, "AI recognition dataset", kaggle, https://www.kaggle.com/datasets/superpotato9/dalle-recognition-dataset

[11]OpenAI, (2022). "DALL·E 2 Preview - Risks and Limitations", GitHub. https://github.com/openai/dalle-2-preview/blob/main/system-card.md

[12]"Seeds", midjourney, https://docs.midjourney.com/hc/en-us/articles/32604356340877-Seeds

[13](2024). "What are diffusion models?", IBM, https://www.ibm.com/think/topics/diffusion-models

감사합니다